UNIVERSIDAD NACIONAL

Escuela de Informática

Programación 4 · II semestre 2026

ENTREGABLE 6

Integración y entrega final del Sistema Web para Gimnasio

Grupo 2 · Repositorio gimnasio-web-grupoN2

Integrantes del grupo: completar con los nombres completos según el acta de constitución.

6 de noviembre de 2026

Repositorio: https://github.com/marjorm/gimnasio-web-grupoN2

Rama de integración: feature/entregable6-integracion-final

Entorno de ejecución documentado: JDK 17, Spring Boot 4.1.1, React, Node.js/Vite y MariaDB.

1. Presentación del proyecto

El Sistema Web para Gimnasio integra una aplicación SPA desarrollada con React, una API Java con Spring Boot y una base de datos relacional MariaDB. Gestiona usuarios, clientes, cursos, rutinas, máquinas, instructores, historial de cursos, pagos, consultas y procesos de inscripción y desinscripción. El acceso a las funciones está delimitado por los roles ADMIN, INSTRUCTOR y CLIENTE.

Este documento consolida la arquitectura, las funcionalidades, las decisiones técnicas, la instalación y las evidencias para el sexto entregable. Se complementa con la documentación técnica, el manual de usuario ilustrado, los archivos SQL y el código fuente en el repositorio.

2. Objetivo y alcance

- Implementar la integración funcional entre la interfaz React, los servicios REST y SOAP de Spring y la persistencia mediante JPA/Hibernate.
- Permitir operaciones de consulta y mantenimiento sobre los recursos solicitados y aplicar reglas de negocio a inscripciones, pagos y desinscripción.
- Proteger las rutas y los datos mediante sesión HTTP, contraseñas BCrypt, roles diferenciados, validaciones y manejo centralizado de excepciones.
- Documentar instalación, arquitectura, patrones, evidencia de ejecución y trabajo colaborativo en GitHub.

3. Arquitectura de la solución

Cliente web (React SPA y React Router) → servidor Node.js/Vite con proxy /api → controladores REST Spring Boot → servicios y reglas de negocio → repositorios JPA/Hibernate → MariaDB. Spring-WS expone, por separado, el ejemplo SOAP del catálogo de cursos mediante WSDL y XSD.

Capa	Tecnología / responsabilidad
Presentación	React, HTML5, CSS3 externo, formularios y navegación SPA
Comunicación	Fetch JSON, API REST y proxy local /api; WSDL/XSD para SOAP
Aplicación	Spring MVC, servicios @Service, validación Bean Validation, sesiones HTTP
Persistencia	JPA/Hibernate y repositorios Spring Data; MariaDB
Servidor web	Node.js/Vite para SPA; Spring Boot para la API
Seguridad	BCrypt, cookies de sesión, 401/403, CORS y cabeceras HTTP

4. Base de datos

El esquema original comprende cliente, usuario, instructores, cursos, máquinas, rutinas e historial_curso. La integración final incorpora estructuras complementarias para inscripciones, programación y cupos, pagos, consultas y bajas; la base de validación gimnasio_entregable6_test alcanzó doce tablas. La configuración usa spring.jpa.hibernate.ddl-auto=none para impedir modificaciones automáticas del esquema.

El script scripts/preparar-entregable6.ps1 configura una base separada de prueba y aplica scripts SQL. El archivo scripts/e6_programacion_demo.sql contiene horarios de demostración, no información comercial oficial del gimnasio.

5. Patrones de diseño obligatorios

Patrón	Implementación / propósito
MVC	Controladores Spring reciben HTTP; servicios resuelven la lógica; modelos JPA representan las entidades; React renderiza la interfaz.
Singleton	Servicios administrados por Spring con alcance singleton para componentes de aplicación compartidos.
Observer	Publicación y observación de eventos de dominio asociados a la creación de cursos.
Iterator	Recorrido de entidades del repositorio mediante Iterator en servicios como MaquinaService y RutinaService.

6. Módulos y funcionalidades

Módulo	Funciones principales
Inicio / navegación	Página principal con imágenes de instalaciones, navegación pública, preferencias de visualización.
Contactos / Blog	Formulario de consulta persistente; enlaces a contenidos externos y redes sociales.
Cursos	Catálogo con código, descripción, programación y disponibilidad; imágenes ilustrativas.
Rutinas	Listados según sesión y rol; acceso del cliente a sus propias rutinas.
Máquinas / Instructores	Información consultada de la base de datos e imágenes uniformadas visualmente.
Inscripciones	Registro persistente de inscripción a cursos o rutinas; control de duplicados y restricciones por cupos.
Desinscripción y pagos	Bloqueo de retiro con pagos pendientes; confirmación de pagos y cancelación.
Mantenimiento	Altas, consultas, modificaciones y bajas de clientes, cursos, máquinas, rutinas e historial; otros apartados administrativos.

7. Autenticación, autorización y seguridad

- Acceso mediante usuario y contraseña; el backend compara las contraseñas con un hash BCrypt.
- La sesión HTTP identifica al usuario; los clientes no pueden listar clientes ajenos ni rutinas de otros.
- ADMIN gestiona configuración y mantenimiento; INSTRUCTOR dispone de operaciones autorizadas según el recurso.
- Validación del lado del cliente y mediante anotaciones @Valid y restricciones del modelo en Spring.
- Control centralizado de excepciones con respuestas comprensibles; errores HTTP 400, 401, 403, 404 y 409 cuando corresponde.
- Persistencia mediante JPA/Hibernate, CORS controlado y cabeceras de seguridad; HSTS corresponde a despliegues HTTPS.

8. Servicios web

La API REST expone los principales recursos bajo /api. Se documentan, entre otras, las rutas /api/usuarios/login, /api/usuarios/sesion, /api/usuarios/logout, /api/clientes, /api/cursos, /api/rutinas, /api/inscripciones y /api/maquinas. La estructura REST utiliza JSON y códigos de estado según el resultado.

El ejemplo SOAP utiliza el contrato disponible en /ws/cursos.wsdl y el esquema XSD /ws/cursosSchema.xsd. Su propósito es evidenciar la diferencia técnica entre servicios basados en contrato SOAP y servicios RESTful.

9. Evidencias de funcionamiento registradas

La evidencia acumulada incluye compilación de la SPA con Vite, análisis de código React, nueve pruebas automatizadas Java aprobadas, pruebas de peticiones HTTP sobre MariaDB y auditoría de navegación de los tres roles. Se conservaron veintiséis capturas del navegador, con cero imágenes rotas y cero errores visibles en las pantallas auditadas.

Las pruebas HTTP registraron respuestas esperadas en login, permisos, inscripción, inscripción duplicada, pago vencido, pago regularizado, cancelación, baja, contacto y CRUD de cursos. Los avisos HTTP 401 observados al consultar una sesión anónima son coherentes con la protección de rutas.

10. Instalación y ejecución

- Instalar Java JDK 17, Maven, Node.js/npm y MariaDB (por ejemplo, XAMPP) en el equipo de desarrollo.
- Clonar el repositorio desde https://github.com/marjorm/gimnasio-web-grupoN2 y seleccionar la rama final del proyecto cuando esté integrada.
- Crear la base de datos de demostración mediante scripts/preparar-entregable6.ps1, sin reemplazar una base de datos existente con registros reales.
- Configurar la conexión de Spring Boot a MariaDB y conservar ddl-auto=none; ajustar las credenciales locales sin publicarlas en GitHub.
- Desde gimnasio-backend/gimnasio-backend, iniciar Spring Boot de acuerdo con el README; desde gimnasio-frontend, instalar dependencias con npm install y ejecutar npm run dev.
- Abrir la dirección local mostrada por Vite (por defecto 5173); el proxy /api conduce las peticiones al backend correspondiente.
- Para entregar y desplegar, usar el código fuente, scripts SQL y README; las cuentas demostrativas no deben ser usadas como cuentas reales.

11. Organización del proyecto entregado

Elemento	Contenido
gimnasio-frontend/	React SPA, componentes, páginas, modelos, servicios, estilos y recursos.
gimnasio-backend/gimnasio-backend/	Spring Boot, modelos, controladores, servicios, repositorios y pruebas.
GIMNASIO.sql	Esquema original y datos iniciales.
GIMNASIO_ENTREGABLE6.sql	SQL consolidado elaborado para la integración.
scripts/	Preparación de MariaDB, migración y programación demostrativa.

documentacion/	Informe, documentación técnica, manual, evidencias y matrices.
README.md	Descripción del proyecto, requisitos y ejecución.

12. Trabajo colaborativo y entrega

El repositorio tiene la rama principal protegida y utiliza Pull Requests. Los PR #17 y #18 quedaron revisados e integrados; el trabajo del entregable final se concentra en feature/entregable6-integracion-final y se incorporará mediante un nuevo PR.

En la entrega institucional debe adjuntarse el enlace al repositorio y la evidencia de GitHub Insights → Contributors, lista de Pull Requests y seguimiento de Issues. El equipo debe incorporar los nombres completos de sus integrantes y el acta correspondiente. La defensa es una actividad posterior coordinada con la profesora.

13. Precisiones del alcance

Se ha utilizado JDK 17 por decisión del equipo. Las fotografías e ilustraciones contenidas en algunas vistas tienen carácter referencial; los horarios incluidos en scripts de demostración no representan necesariamente programación oficial. Los datos de contacto y redes sociales deben corresponder a los perfiles institucionales reales cuando se utilice el sistema fuera del contexto académico.

La documentación describe el proyecto y las evidencias registradas, sin asignar calificación ni sustituir la revisión académica de la profesora.